



AND NETWORK SECURITY

1. Write a C program that contains a string(char pointer) with a value “ Hello World “ The programs should XOR each character in this string with 0 and display the result.

```
#include <stdio.h>
#include <string.h>
main()
{
    printf(" Hello World Program with XOR operation\n");
    char s[]="Hello World";
    char str1[11];
    int i,len;
    len=strlen(s);
    printf("\n The input string as :");
    printf("%s",s);
    printf("\n after XOR string as :");
    for(i=0;i<len;i++)
    {
        str1[i]=s[i]^0;
        printf("%c",str1[i]);
    }
    printf("\n");
    printf(" best of luck");
}
```

```
Hello World Program with XOR operation
```

```
The input string as :Hello World
after XOR string as :Hello World
|best of luck
```

```
=== Code Execution Successful ===
```

2. Write a C program that contains a string (char pointer) with a value \Hello World'. The program should AND or and XOR each character in this string with 127 and display the result.

```
#include <stdio.h>
#include <string.h>
int main()
{
    printf(" Hello World Program with XOR operation\n");
    char s[]="Hello World";
    int i,len;
    len=strlen(s);
    printf("\n The length of string as %d:",len);
    printf("\n The input string as :");
    printf("%s",s);
    printf("\n After AND operation with 127 string as :");
    for (i = 0; i < len; i++)
    {
        printf("%c", s[i] & 127);
    }
    printf("\n\n After XOR with 127 string as :");
    for (i = 0; i < len; i++)
    {
        printf("%c", s[i]^127);
    }
    printf("\n");
    printf(" best of luck");
}
```

Output:

Clear

Hello World Program with XOR operation

The length of string as 11:

The input string as :Hello World

After AND operation with 127 string as :Hello World

After XOR with 127 string as :7????_(???)

best of luck

3. Write a Java program to perform encryption and decryption using the following algorithms: i. Ceaser Cipher ii. Substitution Cipher iii. Hill Cipher

Ceaser Cipher :

```
import java.util.*;
import java.util.Scanner;
class ceaser
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print(" CEASER CIPHER - PROGRAM - SUBSTITUTION ");
        System.out.print("Enter PLIAN text : ");
        String text = sc.next();
        String ptext = text.toUpperCase();
        System.out.print("Enter key: ");
        int pkey = sc.nextInt();
        String cipher = "";
        for (int i = 0; i < ptext.length(); i++) // Encryption
        {
            char ch = ptext.charAt(i);
            if (ch >= 'A' && ch <= 'Z')
                ch = (char) ((ch - 'A' + pkey) % 26 + 'A');
            cipher += ch;
        }
        System.out.println("Encrypted text: " + cipher); // Decryption
        System.out.print(" ");
        System.out.print("Enter CIPHER text : ");
        String text1 = sc.next();
        String ctext = text1.toUpperCase();
        System.out.print("Enter key: ");
        int ckey = sc.nextInt();
        String plain = "";
        for (int i = 0; i < ctext.length(); i++)
        {
            char ch = cipher.charAt(i);
            if (ch >= 'A' && ch <= 'Z')
                ch = (char) ((ch - 'A' - ckey + 26) % 26 + 'A');
            plain += ch;
        }
        System.out.println("Decrypted text: " + plain);
    }
}
```

Output

```
CEASER CIPHER - PROGRAM - SUBSTITUTION
Enter PLIAN text : NEC
Enter key: 3
Encrypted text: QHF

Enter CIPHER text : qhf
Enter key: 3
Decrypted text: NEC
```

ii. Substitution Cipher

```
import java.util.*;
import java.util.Scanner;
public class sub
{
    public static void main(String[] args)
    {
        String alpha = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";
        String key = "QWERTYUIOPASDFGHJKLZXCVBNM";
        Scanner sc = new Scanner(System.in);
        System.out.println("Substitution Cipher - PROGRAM -");
        System.out.println("Enter PLIAN text : ");
        String text = sc.next();
        String ptext = text.toUpperCase();
        String cipher = "";
        for (int i = 0; i < ptext.length(); i++) // Encryption
        {
            char ch = ptext.charAt(i);
            int index = alpha.indexOf(ch);
            cipher += key.charAt(index);
        }
        System.out.println("Encrypted MESSAGE : " + cipher);
        System.out.println();
        String decrypted = "";
        for (int i = 0; i < cipher.length(); i++) // Decryption
        {
            char ch = cipher.charAt(i);
            int index = key.indexOf(ch);
            decrypted += alpha.charAt(index);
        }
        System.out.println("Decrypted MESSAGE : " + decrypted);
    }
}
```

Output

```
Substitution Cipher - PROGRAM -
Enter PLIAN text :
NECcse
Encrypted MESSAGE : FTEELT

Decrypted MESSAGE : NECCSE
|
```

iii. Hill Cipher

```
import java.util.*;
public class hill
{
    static int toNum(char c) // a = 0 mapping
    {
        return c - 'a';
    }

    static char toChar(int n)
    {
        return (char) (n + 'a');
    }
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.println("****HILL CIPHER - PROGRAM:****");
        System.out.print("Enter matrix size n: "); // Read matrix size
        int n = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter plaintext: "); // Read plaintext
        String text = sc.nextLine().toLowerCase().replaceAll("[^a-z]", "");
        while (text.length() % n != 0) // Padding if needed
            text += "x";
        int[][] key = new int[n][n]; // Read key matrix
        System.out.println("Enter key matrix:");
        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                key[i][j] = sc.nextInt();
            }
        }
        StringBuilder cipher = new StringBuilder();
        for (int k = 0; k < text.length(); k += n) // Process blocks
        {
            int[] block = new int[n];
            for (int i = 0; i < n; i++)
                block[i] = toNum(text.charAt(k + i));
            for (int i = 0; i < n; i++) // Multiply block with key
            {
                int sum = 0;
```

```

        for (int j = 0; j < n; j++)
            sum += block[j] * key[j][i];
        cipher.append(toChar((sum % 26 + 26) % 26));
    }
}
System.out.print("The given plaintext : " + text);
System.out.println("");
System.out.println("Enter key matrix:");
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        System.out.print("\t" + key[i][j]);
    }
    System.out.println( "");
}
System.out.println("Ciphertext: " + cipher);
}
}

```

Output

```

***HILL CIPHER - PROGRAM:***
Enter matrix size n: 2
Enter plaintext: ABCD
Enter key matrix:
3 1 5 2
The given plaintext : abcd
Enter key matrix:
3 1
5 2
Ciphertext: fcvi

```

4. Write a Java program to implement the DES algorithm logic.

```
import java.util.Scanner;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

public class des
{
    static byte[] hexToBytes(String hex)// Convert hex string to byte array
    {
        int len = hex.length();
        byte[] data = new byte[len / 2];
        for (int i = 0; i < len; i += 2)
        {
            data[i / 2] = (byte) (
                Character.digit(hex.charAt(i), 16) << 4
                + Character.digit(hex.charAt(i + 1), 16)
            );
        }
        return data;
    }
    static String bytesToHex(byte[] bytes)// Convert byte array to hex string
    {
        StringBuilder sb = new StringBuilder();
        for (byte b : bytes)
            sb.append(String.format("%02X", b));
        return sb.toString();
    }

    public static void main(String[] args) throws Exception
    {
        Scanner sc = new Scanner(System.in);
        System.out.print(" *** DATA ENCRYPTION STANDARDS - PROGRAM *** ");
        System.out.println("Enter plaintext (64-bit Hex): ");
        String plaintextHex = sc.nextLine();

        System.out.print("Enter key (64-bit Hex): ");
        String keyHex = sc.nextLine();

        System.out.print("Enter number of rounds: ");
        int rounds = sc.nextInt();
    }
}
```



```

byte[] data = hexToBytes(plaintextHex);
SecretKey key = new SecretKeySpec(hexToBytes(keyHex), "DES");

Cipher cipher = Cipher.getInstance("DES/ECB/NoPadding");
cipher.init(Cipher.ENCRYPT_MODE, key);
for (int i = 1; i <= rounds; i++) // Apply DES repeatedly for given rounds
{
    data = cipher.doFinal(data);
    System.out.println("After round " + i + ": " + bytesToHex(data));
}
}
}

```

Plain Text : **“NEC EST”**.

Key : CSE

Output

```

*** DATA ENCRYPTION STANDARDS - PROGRAM *** Enter plaintext (6
4E45432045535400
Enter key (64-bit Hex): 4353450000000000
Enter number of rounds: 16
After round 1: 3E5C94EE316B4A83
After round 2: 8EB008E4586E8A39
After round 3: F25D8891ADB2516F
After round 4: CB8DCFB785B9B403
After round 5: A8836D6C351A28A1
After round 6: B587C45D466BE930
After round 7: D5B1A77535B3BFB1
After round 8: E3C3098CEE23918D
After round 9: 16650CCCDF52E5CC
After round 10: DB6FB832BB6EA9E1
After round 11: E22CB12BA6788FEE
After round 12: 2A43D3D59331D5F8
After round 13: 430DB04C7FC91D91
After round 14: BB7920806F5BE142
After round 15: 7364815D3CE245E7
After round 16: AB83D8BCFB59E960

```

AES - PROGRAM

```
import java.util.Scanner;
public class AES
{    // AES S-Box
    static final int[] SBOX =
    {

        0x63,0x7c,0x77,0x7b,0xf2,0x6b,0x6f,0xc5,0x30,0x01,0x67,0x2b,0xfe,0xd7,0xab,0x
        76,0xca,0x82,0xc9,0x7d,0xfa,0x59,0x47,0xf0,0xad,0xd4,0xa2,0xaf,0x9c,0xa4,0x72,
        0xc0,0xb7,0xfd,0x93,0x26,0x36,0x3f,0xf7,0xcc,0x34,0xa5,0xe5,0xf1,0x71,0xd8,0x3
        1,0x15,0x04,0xc7,0x23,0xc3,0x18,0x96,0x05,0x9a,0x07,0x12,0x80,0xe2,0xeb,0x27
        ,0xb2,0x75,0x09,0x83,0x2c,0x1a,0x1b,0x6e,0x5a,0xa0,0x52,0x3b,0xd6,0xb3,0x29,
        0xe3,0x2f,0x84,0x53,0xd1,0x00,0xed,0x20,0xfc,0xb1,0x5b,0x6a,0xcb,0xbe,0x39,0x
        4a,0x4c,0x58,0xcf,0xd0,0xef,0xaa,0xfb,0x43,0x4d,0x33,0x85,0x45,0xf9,0x02,0x7f,
        0x50,0x3c,0x9f,0xa8,0x51,0xa3,0x40,0x8f,0x92,0x9d,0x38,0xf5,0xbc,0xb6,0xda,0x
        21,0x10,0xff,0xf3,0xd2,0xcd,0x0c,0x13,0xec,0x5f,0x97,0x44,0x17,0xc4,0xa7,0x7e,
        0x3d,0x64,0x5d,0x19,0x73,0x60,0x81,0x4f,0xdc,0x22,0x2a,0x90,0x88,0x46,0xee,0
        xb8,0x14,0xde,0x5e,0x0b,0xdb,0xe0,0x32,0x3a,0x0a,0x49,0x06,0x24,0x5c,0xc2,0x
        d3,0xac,0x62,0x91,0x95,0xe4,0x79,0xe7,0xc8,0x37,0x6d,0x8d,0xd5,0x4e,0xa9,0x6
        c,0x56,0xf4,0xea,0x65,0x7a,0xae,0x08,0xba,0x78,0x25,0x2e,0x1c,0xa6,0xb4,0xc6,
        0xe8,0xdd,0x74,0x1f,0x4b,0xbd,0x8b,0x8a,0x70,0x3e,0xb5,0x66,0x48,0x03,0xf6,0
        xe,0x61,0x35,0x57,0xb9,0x86,0xc1,0x1d,0x9e,0xe1,0xf8,0x98,0x11,0x69,0xd9,0x
        8e,0x94,0x9b,0x1e,0x87,0xe9,0xce,0x55,0x28,0xdf,0x8c,0xa1,0x89,0x0d,0xbf,0xe
        6,0x42,0x68,0x41,0x99,0x2d,0x0f,0xb0,0x54,0xbb,0x16

    };

    static String textToHex(String text) // TEXT → HEX (pads to 32 hex chars)
    {
        StringBuilder sb = new StringBuilder();
        for (char c : text.toCharArray())
            sb.append(String.format("%02X", (int) c));
        while (sb.length() < 32)
            sb.append("00");
        return sb.substring(0, 32);
    }

    static int[][] hexToState(String hex) // HEX → AES State
    {
        int[][] state = new int[4][4];
        int k = 0;
        for (int j = 0; j < 4; j++)
        {
            for (int i = 0; i < 4; i++)
            {
                state[i][j] = Integer.parseInt(hex.substring(k, k += 2), 16);
            }
        }
    }
}
```

```

        return state;
    }

static void addRoundKey(int[][] s, int[][] k)
{
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            s[i][j] ^= k[i][j];
        }
    }
}

static void subBytes(int[][] s)
{
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            s[i][j] = SBOX[s[i][j]];
        }
    }
}

static void shiftRows(int[][] s)
{
    int t;
    t=s[1][0]; s[1][0]=s[1][1]; s[1][1]=s[1][2]; s[1][2]=s[1][3]; s[1][3]=t;
    t=s[2][0]; s[2][0]=s[2][2]; s[2][2]=t;
    t=s[2][1]; s[2][1]=s[2][3]; s[2][3]=t;
    t=s[3][3]; s[3][3]=s[3][2]; s[3][2]=s[3][1]; s[3][1]=s[3][0]; s[3][0]=t;
}

static void printState(int[][] s)
{
    for (int i = 0; i < 4; i++)
    {
        for (int j = 0; j < 4; j++)
        {
            System.out.printf("%02X ", s[i][j]);
        }
        System.out.println();
    }
}

```

```

    }
    System.out.println();
}

public static void main(String[] args)
{
    Scanner sc = new Scanner(System.in);
    System.out.println(" **** ADVANCED ENCRYPTION STANDARDS - ADS - Rijndael- PROGRAM: ");
    System.out.println("Enter plaintext (TEXT): ");
    String ptText = sc.nextLine();

    System.out.println("Enter key (TEXT): ");
    String keyText = sc.nextLine();

    System.out.println("Enter number of rounds: ");
    int rounds = sc.nextInt();

    String ptHex = textToHex(ptText);
    String keyHex = textToHex(keyText);

    int[][] state = hexToState(ptHex);
    int[][] key = hexToState(keyHex);

    System.out.println("\nInitial AddRoundKey:");
    addRoundKey(state, key);
    printState(state);

    for (int r = 1; r <= rounds; r++)
    {
        subBytes(state);
        shiftRows(state);
        addRoundKey(state, key);
        System.out.println("After Round " + r + ":");
        printState(state);
    }
}
}

```

Output

```

**** ADVANCED ENCRYPTION STANDARDS - ADS - Rijndael- PROGRAM:
Enter plaintext (TEXT):
NEC BEST
Enter key (TEXT):
CSE
Enter number of rounds:
5

Initial AddRoundKey:
0D 42 00 00
16 45 00 00
06 53 00 00
20 54 00 00

After Round 1:
94 2C 63 63
3D 63 63 47
26 63 6F ED
63 B7 20 63

After Round 2:
61 71 FB FB
A8 FB A0 27

```

Write a Java program to implement RSA Algorithm

```
import java.util.Scanner;

class rsa
{
    static int gcd(int a, int b)
    {
        if (a == 0)
            return b;
        else
            return gcd(b % a, a);
    }

    static int modPow(int base, int exp, int mod)
    {
        int result = 1;
        base = base % mod;
        while (exp > 0)
        {
            if (exp % 2 == 1)
                result = (result * base) % mod;
            base = (base * base) % mod;
            exp = exp / 2;
        }
        return result;
    }

    public static void main(String args[])
    {
        Scanner s = new Scanner(System.in);
        int p, q, n, phi, e = 0, d = 0;
        int msg;
        System.out.println("Enter first prime number p:");
        p = s.nextInt();
        System.out.println("Enter second prime number q:");
        q = s.nextInt();
    }
}
```

AND NETWORK SECURITY

```

n = p * q;
phi = (p - 1) * (q - 1);
System.out.println("Value of n = " + n);
System.out.println("Value of phi(n) = " + phi);
System.out.println("Enter the message (integer < " + n + "):");
msg = s.nextInt();
if (msg >= n || msg < 0)
{
    System.out.println("Encryption not possible (message must be < " + n + "));
    return;
}
char ch = (char) msg; // Convert integer directly to character
System.out.println("Character equivalent = " + ch);
System.out.println("ASCII value = " + msg);
// Find e
for (int i = 2; i < phi; i++)
{
    if (gcd(i, phi) == 1)
    {
        e = i;
        break;
    }
}
System.out.println("Public key (e) = " + e);
// Find d
for (int i = 1; i < phi; i++)
{
    if ((e * i) % phi == 1)
    {
        d = i;
        break;
    }
}

```

AND NETWORK SECURITY

```

System.out.println("Private key (d) = " + d);
int en = modPow(msg, e, n);
System.out.println("Encrypted message = " + en);
char enchar = (char)en;
System.out.println("Encrypted character as = " + enchar);
int de= modPow(en, d, n);
System.out.println("Decrypted message = " + de);
char dechar = (char) de;
System.out.println("Decrypted character = " + dechar);
    }
}

```

Output :

Implement the Diffie-Hellman Key Exchange mechanism using HTML and JavaScript. Consider the end user as one of the parties (Alice) and the JavaScript application as other party (bob).

```
<!DOCTYPE html>
<html>
<head>
  <title>Diffie-Hellman Key Exchange</title>
</head>
<body>

<h2>Diffie-Hellman Key Exchange</h2>

<label>Enter Prime Number (p): </label>
<input type="number" id="p"><br><br>

<label>Enter Primitive Root ( $\alpha$ ): </label>
<input type="number" id="alpha"><br><br>

<label>Enter Alice Private Key (a): </label>
<input type="number" id="a"><br><br>

<button onclick="diffieHellman()">Generate Shared Key</button>

<h3 id="output"></h3>

<script>

function modPow(base, exp, mod) {
  let result = 1;
  base = base % mod;

  while (exp > 0) {
    if (exp % 2 === 1)
      result = (result * base) % mod;
```



```

    base = (base * base) % mod;
    exp = Math.floor(exp / 2);
  }
  return result;
}

function diffieHellman() {

  let p = parseInt(document.getElementById("p").value);
  let alpha = parseInt(document.getElementById("alpha").value);
  let a = parseInt(document.getElementById("a").value);

  // Bob private key (random)
  let b = Math.floor(Math.random() * 10) + 1;

  // Alice public key
  let A = modPow(alpha, a, p);
  // Bob public key
  let B = modPow(alpha, b, p);
  // Shared secrets
  let secretAlice = modPow(B, a, p);
  let secretBob = modPow(A, b, p);
  document.getElementById("output").innerHTML =
    "Bob Private Key (b): " + b + "<br>" +
    "Alice Public Key (A): " + A + "<br>" +
    "Bob Public Key (B): " + B + "<br><br>" +
    "Shared Secret (Alice): " + secretAlice + "<br>" +
    "Shared Secret (Bob): " + secretBob + "<br><br>" +
    "<b>Key Exchange Successful!</b>";
}
</script>
</body>
</html>

```

